

CODE HANDOVER · 코드 설명서

AI-V2X 스마트 지팡이 코드 설명서 (파일별 인수인계)

파이썬 파일 하나하나가 무엇을 하고 · 무엇을 입력받아 · 무엇을 내보내며
RSU 파이프라인의 어느 단계이고 · 다음으로 어떻게 넘어가는지 순서대로

대상: lux/ · scripts/ · AI_Model/ · python/ 의 파이썬 코드
문서 1(시스템편)로 큰 그림을 먼저 잡은 뒤 읽으면 가장 좋습니다.

문서 2 / 2 · 코드편

파일 카드 읽는 법 & 전체 지도

이후 페이지는 파일마다 같은 틀로 설명합니다. 아래 4칸만 기억하면 어떤 파일이든 30초 안에 파악됩니다.

① 무엇을 하는 파일인가

한 줄 요약 + 파이프라인 단계 번호

② 입력 → 출력

어디서 무엇을 받아, 어디로 무엇을 내보내는지

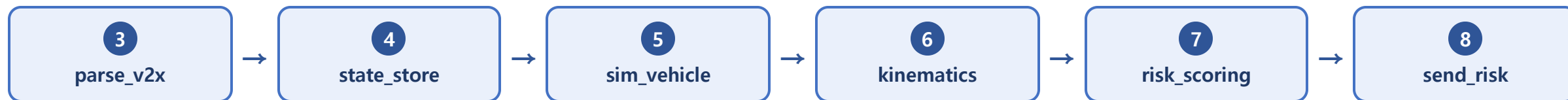
③ 핵심 동작

중요한 함수·로직과 실제 코드 조각

④ 다음 단계로

이 파일의 출력이 어느 파일로 이어지는지

lux/ 실시간 파이프라인 (트랙 A · 주력)



두 트랙 lux/에는 트랙 A(실기 검증된 주력 step 파이프라인)와 트랙 B(SUMO/AI 통합용 차세대 융합 엔진)가 함께 있습니다. 먼저 트랙 A를 순서대로 보고(4~9p), 트랙 B는 10p에서 정리합니다.

왜 lux/에 코드가 두 계보인가

헛갈리지 않도록 먼저 구분합니다. 위험도를 만드는 철학이 서로 다릅니다.

트랙 A · step 파이프라인 (주력 ★)

parse_v2x → state_store → sim_vehicle → kinematics →
risk_scoring → send_risk

Jetson에서 실기 검증까지 끝난 라인. 팀 점수표(0~100) + 칼만 필터 + DCPA 게이트로 risk 0~3 산출. 각 파일은 Jetson에 stepN_*.py로 배포됨.

트랙 B · risk_engine 융합 (차세대)

risk_engine + jetson_rsu_bridge + zone_risk + fusion +
predict_risk

SUMO/AI(Transformer·ONNX) 통합을 겨냥한 실험 계열. **rule + zone + AI**를 max()로 융합해 0~3 산출. 좌표계 통합 계약이 아직 확정 전.

두 트랙의 공통점

둘 다 "거리·TTC로 위험을 판단하고, 안전측(높은 위험)을 택한다"는 원칙은 같습니다. 트랙 A는 점수표, 트랙 B는 3소스 max — 방식만 다릅니다. 이 문서의 RSU 파이프라인 설명은 트랙 A 기준입니다.

참고 각 *.py마다 짝이 되는 test_*.py가 있습니다(단위 테스트 다수). 코드를 고칠 때는 해당 테스트를 먼저 돌려 보세요.

정의 코드 vs 실행 코드 — 파일로 구분하기

최민서 파트와 내 파트가 겹쳐 보이는 이유는 같은 "risk"라는 말을 쓰기 때문입니다. 실제로는 기준을 만드는 코드와 그 기준으로 실시간 판정하는 코드로 갈립니다.

최민서 — 정의 코드 (오프라인 · 가끔 실행)

scripts/risk_calculator.py — 점수표 원본(정의)
scripts/zone_detector.py — 위험구역 정의
scripts/event_analyzer.py — 라벨 부여
AI_Model/.../train_transformer.py — 모델 학습
AI_Model/.../export_onnx.py — .onnx 산출

= 기준·라벨·모델 "파일"을 만든다

나 — 실행 코드 (실시간 · 현장에서 상시) ★

lux/parse_v2x · state_store — 수신·상태
lux/kinematics — 거리·TTC 계산
lux/risk_scoring — 점수 적용·레벨 결정
lux/zone_risk · predict_risk — zone 판정·ONNX 추론 실행
lux/fusion — max 융합(최종 결정)
lux/send_risk — 지팡이로 전송

= 그 파일들을 받아 실시간 판정한다

핵심 — 최종 risk를 "결정"하는 코드는 전부 lux/ 안에 있다

모델(.onnx)과 점수표는 최민서가 만들지만, 그것을 실행해 위험 등급을 정하고 지팡이를 올리는 코드는 모두 내 lux/입니다. AI는 rule을 대체하지 않고 rule 산출물(ttc·risk_score·zone)을 입력으로 먹습니다.

⚠ 주의 같은 점수표가 scripts/risk_calculator.py(정의 원본)와 lux/risk_scoring.py(실행 복사본)에 이중 존재 → 단일 소스로 관리 필요.

parse_v2x.py

파이프라인의 입구. RSU가 USB로 보낸 줄 단위 V2X JSON을 파싱해, 출처를 표시하고, 표준 17컬럼 형식으로 CSV에 기록합니다.

INPUT

시리얼 /dev/ttyUSB0 (115200) 줄 단위 JSON — RSU ESP32가 USB로 한 줄에 객체 하나씩. (또는 --stdin)

OUTPUT

step3_v2x_parsed_log.csv — 17개 고정 컬럼 행을 append. 콘솔에 파싱 로그.

핵심 동작

- SOURCE_MODES = real·test·fallback·simulation — 데이터 출처 4종
- normalize_record() — payload를 17컬럼 표준 행으로. payload에 source_mode가 있으면 CLI 기본값보다 우선(cane/vehicle 혼합 스트림 대응)
- serial_lines() — 포트 열고 reset_input_buffer()로 부팅 잡음 제거 후 줄 단위 yield. 다른 step 파일이 이 함수를 재사용

```
# 출처는 gps_valid가 아니라 source_mode로 판단
if embedded_mode in SOURCE_MODES:
    row["source_mode"] = embedded_mode # payload 우선
else:
    row["source_mode"] = cli_default

# 17컬럼: pc_time,type,node_id,seq,gps_valid,
# lat,lng,speed_mps,heading_deg,node_risk,...
```

다음 단계로 →

표준화된 한 행이 **state_store.py(4단계)**로 전달되어, 차량·지팡이의 "최신 상태"로 보관됩니다.

state_store.py

최신 상태 보관소. 차량·지팡이의 가장 최근 레코드를 타입별로 들고 있다가, 위험도를 계산할 수 있는 "쌍"이 준비됐는지 판정합니다.

INPUT

parse_v2x와 같은 JSON 스트림. --test-vehicle이면 sim_vehicle의 가짜 차량을 같은 루프에서 주입.

OUTPUT

콘솔 스냅샷 + pair_valid(둘 다 신선한가) / risk_valid(둘 다 좌표 있는가). CSV 없음.

핵심 동작

- has_position() — gps_valid가 아니라 좌표값 자체로 판정. 실내 fallback 좌표(37.0/127.0)도 통과시켜 개발 가능
- StateStore.status() — MISSING / STALE / READY (신선도 창 FRESH_WINDOW_S=0.5s)
- snapshot() → (상태, pair_valid, risk_valid)

```
# 0.5초 안에 들어온 것만 "신선(READY)"
def status(self, kind, now):
    if kind not in self.latest: return "MISSING"
    age = now - self.latest[kind].t
    return "READY" if age <= 0.5 else "STALE"
```

다음 단계로 →

risk_valid가 참이면(차량·지팡이 좌표가 모두 신선) **kinematics.py(6단계)**가 이 StateStore를 물려받아 상대 운동을 계산합니다.

sim_vehicle.py

가짜 차량 생성기 (테스트용). 실제 차량 ESP32 신호가 아직 없을 때, 지팡이 쪽으로 점점 접근하는 가상 차량을 만들어 6~8단계를 개발·검증하게 합니다.

INPUT

표적(지팡이) 좌표 — 파이프라인 내부에서는 StateStore의 최신 cane 좌표를 겨냥.

OUTPUT

type:"vehicle", source_mode:"simulation"이 박힌 JSON + 의도 거리(distance_m).

핵심 동작

- 상수: START_DISTANCE=50m, SPEED=5m/s, PERIOD=0.2s(5Hz), 정면 접근
- offset_position() — 표적에서 지정 방향·거리만큼 떨어진 점 계산
- TestVehicle.record() — 거리가 0에서 멈춰 단조 감소 보장(점점 다가옴)
- 거리 상수를 kinematics와 일부러 공유 → "생성 거리"와 "측정 거리"가 일치하는지로 수식 검증

```
# 표적으로 5m/s씩 접근, 거리는 0에서 멈춤
def distance_at(self, elapsed):
    d = START_DISTANCE - SPEED * elapsed
    return max(0.0, d) # 단조 감소
```

9단계 예정: 이 파일을 실차 ESP32 신호로 교체합니다. --test-vehicle 플래그만 빼면 되고, 6~8단계 코드는 손대지 않습니다.

다음 단계로 →

주입된 가짜 차량 레코드가 **state_store** → **kinematics**로 흘러 실제 차량처럼 처리됩니다.

kinematics.py

운동학 계산기. 차량-지팡이의 거리·접근속도·TTC·CPA를 raw(측정값)와 칼만 필터(평활) 두 갈래로 계산합니다.

INPUT

StateStore의 최신 cane/vehicle 좌표·속도·헤딩.

OUTPUT

step6_kinematics_log.csv (raw/filtered 14필드) + filtered 트랙(다음 단계 입력).

핵심 동작

- LocalFrame — 첫 cane 좌표를 원점으로 하는 로컬 ENU(미터) 평면. 위경도로는 벡터 내적을 못 하므로 필수
- relative_kinematics() → distance, closing(접근속도), **ttc**, tcpa, **dcpa**
- KalmanCV1D — 등속도 칼만을 east/north 2개로 분해 → **numpy 없이** Jetson에서 동작

```
# 접근할 때만 TTC, 아니면 안전값
closing = -d(distance)/dt
if closing > 0:
    ttc = distance / closing
else:
    ttc = 999.0 # 멀어지는 중

# dcpa = 최근접시 거리(옆 스침 판별용)
```

다음 단계로 →

잡음이 평활된 **filtered 트랙**(distance·ttc·dcpa)이 **risk_scoring.py(7단계)**로 전달되어 점수가 매겨집니다.

risk_scoring.py

위험도 채점기 — 여기서 실시간 risk 등급이 정해집니다. 점수표 정의는 **최민서(scripts/risk_calculator.py)**, 실행·판정은 이 파일(나). 옆으로 스치는 차를 억제하는 **DCPA 게이트**는 이 파일이 추가한 부분입니다.

INPUT

kinematics의 filtered 값(distance·ttc·rel_speed·dcpa) + 차량 속도.

OUTPUT

step7_risk_log.csv (base_score, gate, final_score, **risk_level** 등).

핵심 동작

- `calculate_risk_score()` — 거리(≤ 30) + TTC(≤ 35) + 상대속도(≤ 20) + 차량속도(≤ 10) + zone(≤ 5), 상한 100 (scripts/risk_calculator.py 점수표 그대로)
- `classify_risk_level()` — $\geq 70 \rightarrow 3$, $\geq 45 \rightarrow 2$, $\geq 20 \rightarrow 1$, 그 외 0
- `dcpa_gate()` — near 2.5m / far 7.5m / floor 0.2. 옆 스침이면 배율을 낮춰 위험도 억제

```
# 점수 × DCPA 게이트 = 최종
base = calculate_risk_score(dist, ttc, ...)
gate = dcpa_gate(dcpa) # 0.2 ~ 1.0
final = base * gate
level = classify_risk_level(final)
# dcpa=None(멀어짐)이면 gate=1.0(투명)
```

다음 단계로 →

최종 **risk_level(0~3)**이 **send_risk.py(8단계)**로 넘어가 RSU로 전송됩니다.

send_risk.py

파이프라인의 출구. risk_level을 RSU 다운링크로 내보내되, 신뢰 게이팅과 전송 레이트 제한을 적용해 근거 없는 진동과 과도한 전송을 막습니다.

INPUT

시리얼 스트림(같은 포트에서 read+write) 또는 --stdin. 7단계 채점을 import해 자체 수행.

OUTPUT

시리얼로 {"target_id":0,"risk":N} 다운링크 + step8_risk_tx_log.csv.

핵심 동작

- RiskTransmitter.consider() — 순수 상태머신(시리얼 없이 테스트 가능)
- **트러스트 게이팅**: GPS 신뢰 안 되면 effective=0으로 낮춰 **경보 해제만 허용**(근거 없는 진동 차단). --tx-untrusted로 우회
- **레이트 제한**: 값이 바뀌면 즉시(change), 아니면 1초 주기(heartbeat), 그 외 보류(hold)

```
# 신뢰 안되는 GPS면 위험 경보를 억제
if not gps_trusted:
    effective = 0 # 해제만 허용
# 변화 시 즉시 / 아니면 1초마다
if level != last or now-last_t >= 1.0:
    send({"target_id":0, "risk": effective})
```

다음 단계로 →

RSU가 target_id=0을 받으면 **본 모든 단말에 브로드캐스트** → 지팡이가 진동·부저 출력. 여기서 실시간 파이프라인 한 바퀴가 끝납니다.

트랙 B — 최종 판정(max)이 일어나는 자리

rule + zone + AI 세 위험도를 max()로 합치는 계열입니다. **최종 risk를 결정하는 max 융합과 ONNX 추론 실행이 모두 여기(=내 코드, Jetson)에서** 일어납니다 — SUMO나 서버가 아닙니다.

파일	역할 · 입출력
risk_engine.py	코어 계산 엔진. RiskInput→RiskResult. haversine 거리, rule 위험도, zone 위험도, merge_risks()=3소스 max
jetson_rsu_bridge.py	RSU ESP32 ↔ Jetson 시리얼 브리지 . cane 상태 유지, vehicle마다 evaluate_risk() → JSON 응답 회신 (트랙 A의 parse+send를 한 파일로)
zone_risk.py	보행자 (x,y)를 정적 위험구역에 매핑 → 0~3 레벨. zone_definition.csv 사용
fusion.py ★	<code>fuse_risk(rule, zone, predicted) = max</code> — 최종 risk 등급이 확정되는 지점 . 한 갈래가 놓쳐도 다른 갈래가 잡는 안전 우선 구조
predict_risk.py	Transformer(ONNX) 추론 실행부 . 모델 파일은 최민서 제작, 실행은 여기 . 모델 없으면 None으로 graceful degrade

지금 상태

트랙 B는 좌표계 통합 계약(docs/integration_contract.md)이 확정되면 트랙 A의 위험 산정부를 보강할 **후보**입니다. 현재 AI 예측 자리는 transformer_risk_placeholder(항상 0)로, ONNX 모델을 붙이면 실제 예측이 들어갑니다. probe_risk_downlink.py는 다운로드 경로 생존만 확인하는 일회성 실험입니다.

scripts/run_scenario.py

한 시나리오를 자동 처리하는 지휘자. feature.csv를 받아 rule 위험도와 이벤트 라벨을 한 번에 생성합니다.

INPUT

시나리오명(예 scenario_000)과 그 폴더의 feature.csv.

OUTPUT

시나리오 폴더에 risk_result.csv + event_result.csv.

1

feature.csv를
jetson_run으로 복사



2

main.py 실행
(rule 위험도)



3

risk_result.csv
시나리오로 복사



4

event_analyzer
실행 → 라벨

핵심 동작

내부에서 jetson_run/main.py(rule 추론)와 event_analyzer.py(라벨링)를 순서대로 호출합니다.

⚠ 주의: 4번 단계에서 event_analyzer.py의 입출력 경로 줄을 **실행 중에 문자열 치환으로 덮어씁니다**. 그래서 파일 상단 경로가 마지막 실행 시나리오(scenario_005)로 남아 있습니다. 재실행 시 유의하세요.

다음 단계로 →

각 시나리오의 event_result.csv가 만들어지면 **merge_dataset.py(13p)**가 이들을 하나로 합칩니다.

라벨링 4형제 — 정답을 만든다

event_analyzer.py가 나머지 3개를 불러 각 행에 zone·위험도·이벤트유형을 붙입니다. 이 결과가 AI의 **정답 라벨**이 됩니다.

event_analyzer.py

지휘자. 각 행마다 보행자·차량 중간점을 이벤트 위치로 잡고, 아래 3개를 호출해 **22컬럼 event_result.csv** 생성.

risk_calculator.py

거리·TTC·상대속도·차량속도·zone → **0~100점** → **risk_level($\geq 70/45/20$)**.
Transformer가 학습하는 기준(방식 A).

zone_detector.py

SUMO 좌표(x,y)가 등록 위험구역(반경 30m) 안인지 판정.
zones/zone_definition.csv(정문·중문·학생회관·주차장출구).

event_classifier.py

사람이 읽는 이벤트 유형 문자열: Near Miss / Parking Exit / Blind Spot /
Vehicle Yield / Safe Pass / ...

결과 — event_result.csv (22컬럼)

feature 9컬럼 + event_id · event_x · event_y · ttc · risk_score · risk_level · zone_id · zone_name · zone_type · zone_distance_m · zone_base_risk · zone_speed_limit_kmh · event_type

다음 단계로 →

시나리오별 라벨 CSV가 **merge_dataset.py**로 모여 통합 데이터셋이 됩니다.

데이터를 하나로 모은다

흩어진 시나리오 결과를 합쳐 AI 학습에 바로 넣을 수 있는 한 파일로 만듭니다.

scripts/merge_dataset.py

여러 시나리오 → master

모든 scenario_*/event_result.csv를 세로로 concat하고, 맨 앞에 scenario 컬럼과 label 컬럼을 추가.

OUTPUT

dataset/master_dataset.csv (24컬럼)

AI_Model/.../merge_labels.py

피쳐 + 라벨 안전 병합

피쳐 파일과 라벨 파일을 ts_ms·좌표 5키로 np.allclose 검증 후 병합 — 다른 SUMO 결과가 섞이는 것을 방지.

OUTPUT

labeled_master_dataset.csv (23컬럼, 학습 입력)

⚠ 재현성 주의

현재 master_dataset.csv(약 7.9만 행)와 labeled_master_dataset.csv(약 1.26만 행)의 행 수가 다릅니다. merge_labels는 두 입력 행 수가 같아야 하므로, 지금 학습 데이터는 **과거 단일 시나리오 시점**에 만들어진 것으로 보입니다. 재현하려면 버전 정합이 필요합니다.

다음 단계로 →

labeled_master_dataset.csv가 **train_transformer.py(14p)**의 학습 입력이 됩니다.

AI_Model/transformer/train_transformer.py

Transformer 위험도 분류기 학습. 라벨된 CSV로 최근 10프레임을 보고 현재 위험 등급(0~3)을 맞추도록 학습합니다.

INPUT

labeled_master_dataset.csv — 11개 피쳐 + risk_level 라벨.

OUTPUT

risk_transformer.pt + scaler.json + training_report.txt.

핵심 동작

- 피쳐 11개(순서 고정): ped_x/y, veh_x/y, ped-veh 속도, distance, rel_speed, **ttc**, **risk_score**, **zone_base_risk**
- StandardScaler로 z-score 정규화 → mean/scale를 scaler.json에 저장
- SEQUENCE_LENGTH=10 슬라이딩 윈도우, 마지막 프레임 라벨이 정답
- 클래스 불균형 보정 가중치 적용, val_loss 최소 시점 저장

```
# 입력 투영 → Transformer → 마지막 프레임 분류
x = Linear(11→64)(x)
x = TransformerEncoder(d=64,h=4,L=2)(x)
x = x[:, -1, :] # 마지막 타임스텝
logits = Linear(64→4)(x) # 4클래스
```

주의: 피쳐 뒤 3개(ttc-risk_score-zone_base_risk)는 **rule 산출물** — 추론 시에도 rule을 먼저 계산해 넣어야 합니다.

다음 단계로 →

학습된 .pt를 **export_onnx.py(15p)**가 ONNX로 변환합니다.

모델을 엣지로 옮기고 확인한다

export_onnx.py

.pt → .onnx 변환

학습된 PyTorch 모델을 ONNX로 변환. 더미 입력 [1,10,11], opset 17, batch 동적축. 입력명 input / 출력명 logits [batch,4].

왜 ONNX? PyTorch 없이 경량 onnxruntime으로 Jetson·엣지에서 CPU 추론하기 위함. 크로스 플랫폼·경량 배포.

test_onnx.py

추론 스모크 테스트

scaler.json의 mean/scale로 **수동 z-score** → 앞 10행을 [1,10,11]로 → onnxruntime(CPU) 추론 → argmax = 예측 Level.

중요: 학습은 sklearn StandardScaler, 추론은 scaler.json 값으로 **수동 정규화** — 추론 측이 반드시 동일 정규화를 재현해야 결과가 맞습니다.

merge_labels.py

피처+라벨 5키 검증 병합(13p에서 설명).

training_report.txt

테스트 정확도 99.3% (단, Near Miss 6샘플 불균형).

scaler.json

feature별 mean/scale + sequence_length. 추론 필수.

다음 단계로 →

변환·검증된 .onnx는 트랙 B의 **predict_risk.py**에 실려 실시간 예측에 쓰입니다(현재 연결은 확장 단계).

python/jetson_run/ — 현장 rule 추론

현재 Jetson 실행부는 거리 기반 rule로 위험도를 계산합니다(AI 추론은 아직 미연결). SUMO 데이터 처리(run_scenario)에서도 이 모듈을 호출합니다.

main.py

진입점. 배너 출력 후 risk_inference.run() 호출만 합니다.

risk_inference.py

거리 기반 등급(방식 B): $<3 \rightarrow 3$, $<10 \rightarrow 2$, $<30 \rightarrow 1$, else 0. $TTC = \text{distance}/\text{rel_speed}$.
입력 feature_9cols.csv → 출력 output/risk_result.csv(+ttc, risk_level).

⚠ 위험도 정의가 두 벌

방식 A (risk_calculator: 점수표 $\geq 70/45/20$) — Transformer 학습 기준

방식 B (risk_inference: 거리 $< 3/10/30$) — Jetson 실시간

실시간에 AI를 붙이려면 방식 A로 통일이 필요합니다 (integration_contract Part 3).

feature_9cols.csv = SUMO feature와 같은 9컬럼 Jetson 입력.
run_scenario가 시나리오 파일을 이 위치로 복사해 넣습니다.

docs/integration_contract.md — 꼭 정독

세 모듈을 붙일 때의 좌표계·feature·위험도 정의 합의서. AI 입력 규격([batch, 10, 11]), 좌표계 3종 혼재 경고, risk 정의 2벌 문제가 모두 정리되어 있습니다.

한 파일씩, 이 순서로 돈다

왼쪽은 기준·모델을 만드는 오프라인(최민서), 오른쪽은 그것으로 실시간 판정하는 온라인(나). 두 갈래는 **모델 파일(.onnx)**에서 만납니다.

오프라인 · 정의/학습 (최민서)

순	파일
1	SUMO → feature.csv (9컬럼)
2	run_scenario.py
3	event_analyzer(+risk_calculator/zone/classifier)
4	merge_dataset.py → master
5	merge_labels.py → labeled
6	train_transformer.py → .pt
7	export_onnx.py → .onnx (test_onnx로 검증)

실시간 · 판정/실행 (나) ★

순	파일
3	parse_v2x.py – RSU JSON 파싱
4	state_store.py – 최신 상태·쌍 판정
5	sim_vehicle.py – (실차 전) 가짜 차량
6	kinematics.py – 거리·TTC·칼만
7	risk_scoring.py – 점수·DCPA → 0~3 결정
+	predict_risk(ONNX 실행) · zone_risk · fusion(max)
8	send_risk.py – RSU로 전송 → 지팡이

최종 risk가 확정되는 지점은 7단계와 fusion(max) — 둘 다 내 코드입니다. 오프라인이 만든 .onnx는 predict_risk가 불러 이 max에 한 표로 들어갑니다.

코드 지도, 이제 손에 들었습니다

어떤 파일을 열어도 "어느 단계, 다음은 어디"가 보인다

실시간 파이프라인은 `parse` → `state` → `kinematics` → `risk` → `send` 순서로,
데이터/AI 파이프라인은 `SUMO` → 라벨링 → 병합 → 학습 → `ONNX` 순서로 흐릅니다.

막히면 각 파일의 `test_*.py`를 먼저 돌리고, 규격이 헛갈리면 `integration_contract.md`와
`v2x_session_summary_next_steps.md`를 펴 보세요.